

Arrays and Strings in Bash Scripting

Arrays in Bash Scripting

- In Bash scripting, arrays are powerful tools for managing lists of elements or values.
- Unlike some programming languages, Bash arrays are always one-dimensional, but they can still be quite useful.

Declaring an Array

In Bash, arrays can be declared with or without specifying elements initially.

Empty array declaration:

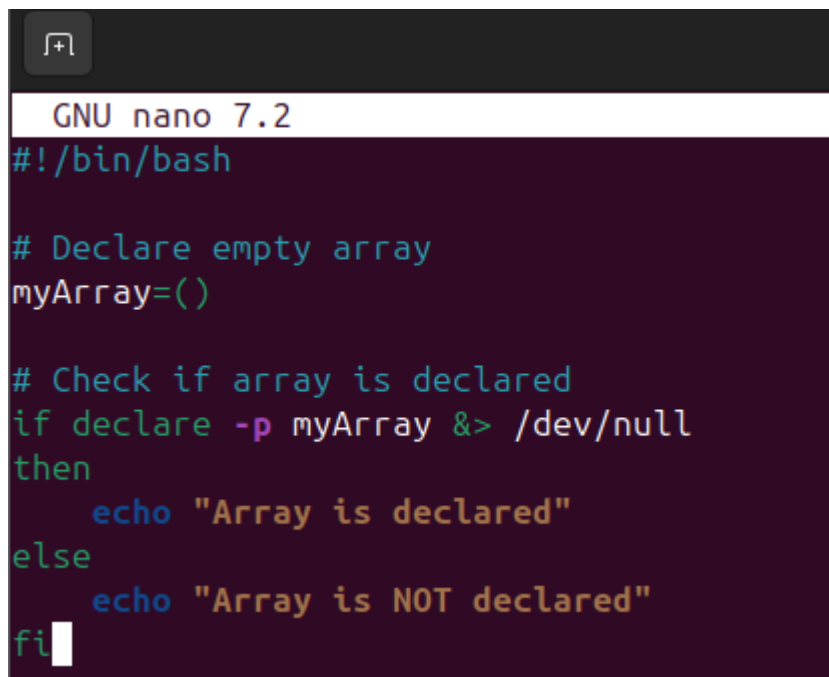
```
declare -a array_name
```

Array with elements:

```
array_name=(value1 value2 value3)
```

Practice Questions

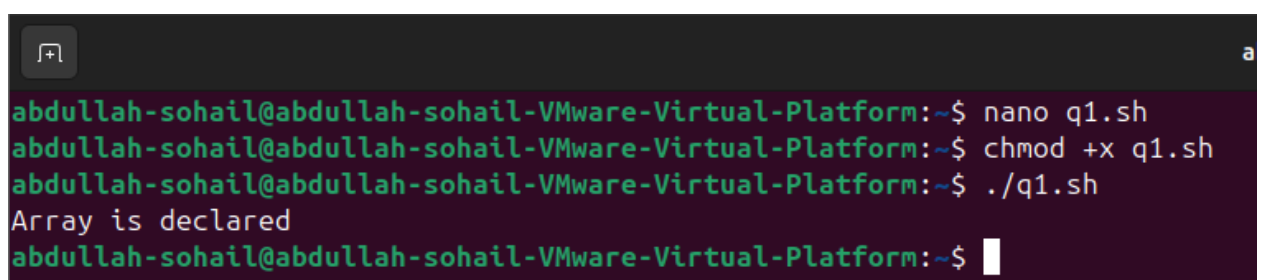
- 1) How can you verify if an array has been declared in Bash, even if it is empty?



```
GNU nano 7.2
#!/bin/bash

# Declare empty array
myArray=()

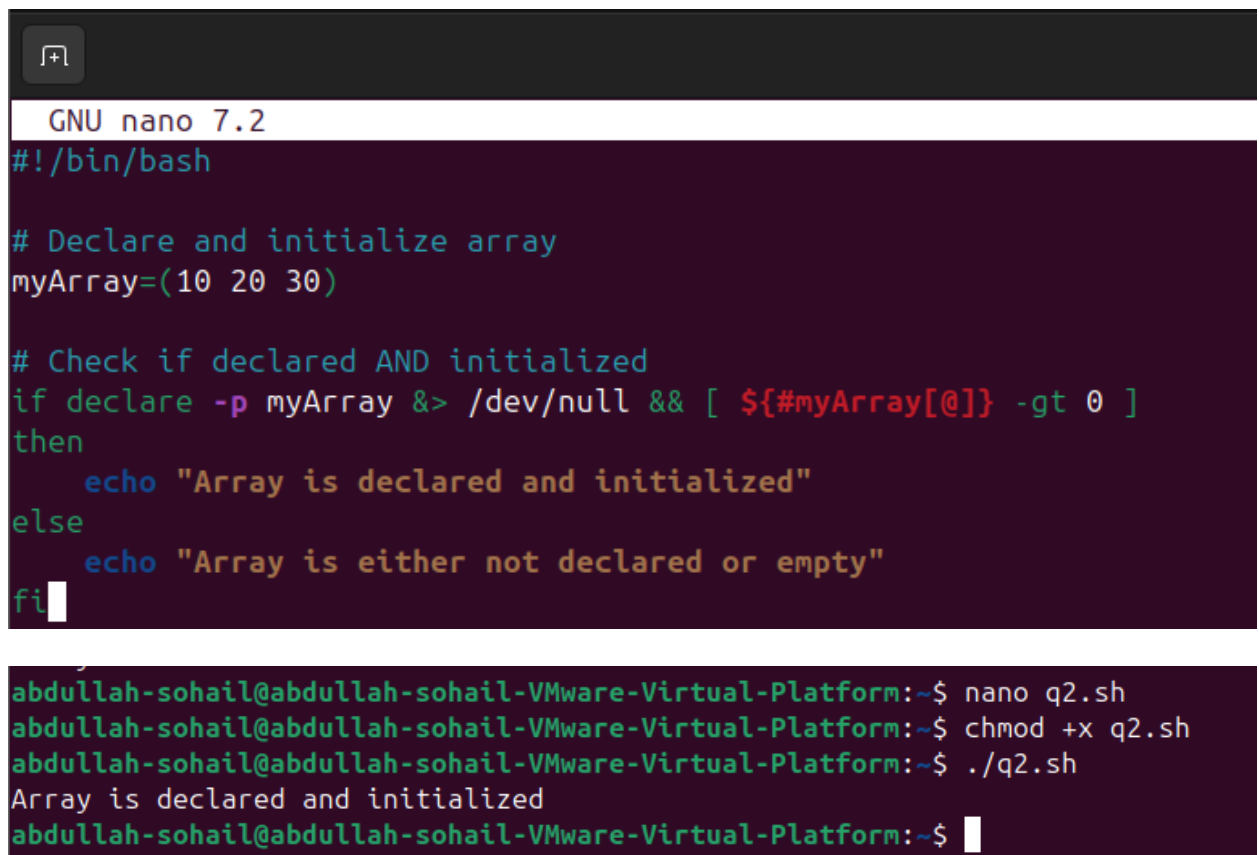
# Check if array is declared
if declare -p myArray &> /dev/null
then
    echo "Array is declared"
else
    echo "Array is NOT declared"
fi
```



```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano q1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x q1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./q1.sh
Array is declared
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

The University of Lahore

- 2) How can you check if an array has been both declared and initialized (i.e., contains at least one element) in Bash?



```
GNU nano 7.2
#!/bin/bash

# Declare and initialize array
myArray=(10 20 30)

# Check if declared AND initialized
if declare -p myArray &> /dev/null && [ ${#myArray[@]} -gt 0 ]
then
    echo "Array is declared and initialized"
else
    echo "Array is either not declared or empty"
fi

abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano q2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x q2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./q2.sh
Array is declared and initialized
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

Accessing Array Elements

To access individual elements in an array, use the `${array_name[index]}` syntax.

```
echo ${array_name[0]} # Access the first element
echo ${array_name[1]} # Access the second element
```

Access all elements:

```
echo ${array_name[@]} # Prints all elements in the array
echo ${array_name[*]} # Alternative to @, also prints all
elements
```

Adding and Modifying Elements

You can add or update elements by assigning a value to a specific index.

```
array_name[3]="new_value" # Adds or updates the fourth element
```

The University of Lahore

Array Length

To find the number of elements in an array:

```
echo ${#array_name[@]} # Prints the number of elements
```

Looping Through Arrays

Use a `for` loop to iterate over elements in an array.

```
for element in "${array_name[@]"; do
    echo $element
done
```

Or loop through indices:

```
for index in "${!array_name[@]"; do
    echo "Element at index $index: ${array_name[index]}"
done
```

Removing Elements

Use the `unset` command to remove an element at a specific index.

```
unset array_name[1] # Removes the second element
```

To clear the entire array:

```
unset array_name
```

Associative Arrays

Associative arrays (key-value pairs) are available in Bash 4.0 and above.

```
declare -A assoc_array
assoc_array[key1]="value1"
assoc_array[key2]="value2"
# Accessing elements
echo ${assoc_array[key1]}
# Looping through associative arrays
for key in "${!assoc_array[@]"; do
    echo "Key: $key, Value: ${assoc_array[$key]}"
done
```

Example

Here's a full example of working with arrays in Bash:

```
# Declare and initialize an array
fruits=("Apple" "Banana" "Cherry")

# Add an element
fruits[3]="Date"

# Print all elements
echo "All fruits: ${fruits[@]}"

# Print length of the array
echo "Number of fruits: ${#fruits[@]}"

# Loop through elements
for fruit in "${fruits[@]"; do
    echo "Fruit: $fruit"
done

# Remove an element
unset fruits[1] # Removes "Banana"

# Print all elements after removal
echo "Fruits after removal: ${fruits[@]}"
```

Practice Questions

- 1) Write a Bash script that stores the names of all `.txt` files in a directory into an array and then prints the array.

The University of Lahore

```
GNU nano 7.2
#!/bin/bash

# Store all .txt files in an array
txtFiles=(*.txt)

# Print array
echo "All .txt files:"
for file in "${txtFiles[@]"; do
    echo "$file"
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Qu1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Qu1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Qu1.sh
All .txt files:
important.txt
input.txt
output.txt
processes.txt
test1.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 2) Write a script that stores the names of all `.txt` files in an array and then counts the number of lines in each file.

```
GNU nano 7.2
#!/bin/bash

txtFiles=(*.txt)

for file in "${txtFiles[@]"; do
    if [ -f "$file" ]; then
        lines=$(wc -l < "$file")
        echo "$file has $lines lines"
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Qu2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Qu2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Qu2.sh
important.txt has 1 lines
input.txt has 3 lines
output.txt has 977 lines
processes.txt has 335 lines
test1.txt has 1 lines
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

The University of Lahore

- 3) Write a script that stores the names of all `.txt` files in an array and then finds files larger than a specified size (e.g., 1000 bytes).

```
GNU nano 7.2
#!/bin/bash

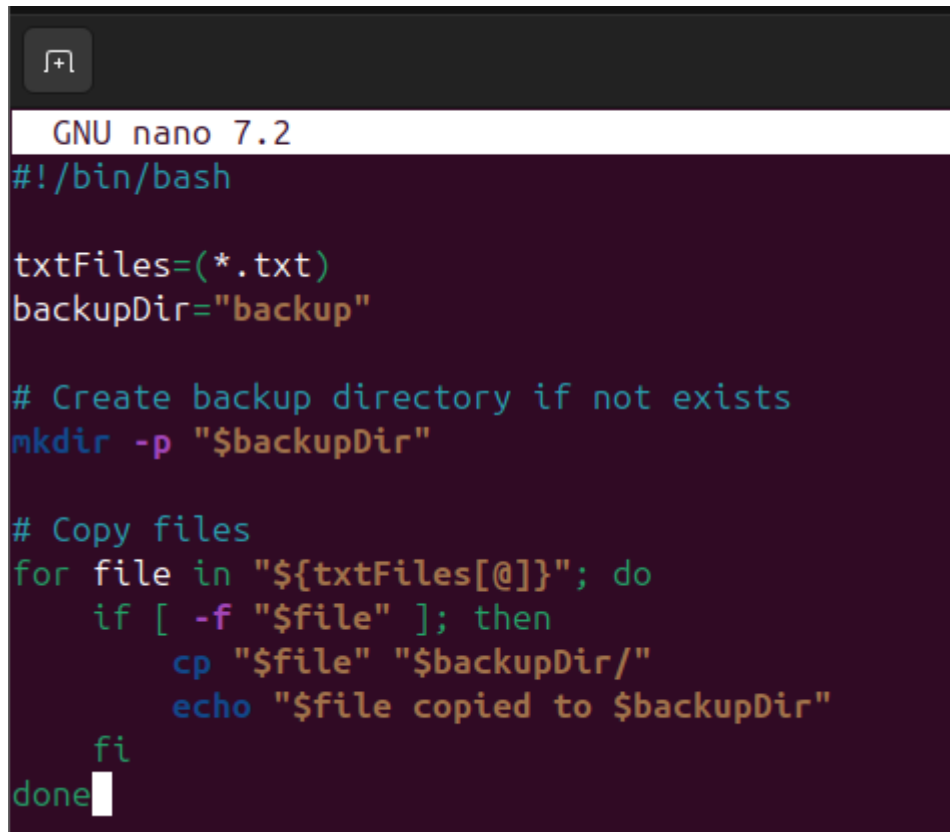
txtFiles=(*.txt)
sizeLimit=1000 # in bytes

for file in "${txtFiles[@]"; do
    if [ -f "$file" ] && [ $(stat -c%s "$file") -gt $sizeLimit ]; then
        echo "$file is larger than $sizeLimit bytes"
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Qu3.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Qu3.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Qu3.sh
output.txt is larger than 1000 bytes
processes.txt is larger than 1000 bytes
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

The University of Lahore

- 4) Write a script that stores the names of all `.txt` files in an array and copies them to a backup directory.



```
GNU nano 7.2
#!/bin/bash

txtFiles=(*.txt)
backupDir="backup"

# Create backup directory if not exists
mkdir -p "$backupDir"

# Copy files
for file in "${txtFiles[@]"; do
    if [ -f "$file" ]; then
        cp "$file" "$backupDir/"
        echo "$file copied to $backupDir"
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Qu4.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Qu4.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Qu4.sh
important.txt copied to backup
input.txt copied to backup
output.txt copied to backup
processes.txt copied to backup
test1.txt copied to backup
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 5) Write a script that stores the names of all `.txt` files in an array and lists those that contain more than 100 words.

```
GNU nano 7.2
#!/bin/bash

txtFiles=(*.txt)

for file in "${txtFiles[@]}"; do
    if [ -f "$file" ]; then
        wordCount=$(wc -w < "$file")
        if [ "$wordCount" -gt 100 ]; then
            echo "$file has more than 100 words ($wordCount words)"
        fi
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Qu5.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Qu5.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Qu5.sh
output.txt has more than 100 words (11673 words)
processes.txt has more than 100 words (4011 words)
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 6) Write a script that stores the names of all `.txt` files in an array and deletes those that are empty.

```
GNU nano 7.2
#!/bin/bash

txtFiles=(*.txt)

for file in "${txtFiles[@]}"; do
    if [ -f "$file" ] && [ ! -s "$file" ]; then
        rm "$file"
        echo "$file was empty and deleted"
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Qu6.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Qu6.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Qu6.sh
```


The University of Lahore

Strings in Bash Scripting

- In Bash scripting, strings are one of the most common data types you'll work with.
- Here's an overview of string handling in Bash, from basic to advanced concepts:

Basic String Assignment

Assigning a String to a Variable:

```
name="John"
echo $name
```

Concatenating Strings:

```
first_name="John"
last_name="Doe"
full_name="$first_name $last_name"
echo $full_name
```

Accessing String Length

Getting the Length of a String:

```
str="Hello, world!"
echo ${#str} # Outputs: 13
```

String Comparison

Comparing Strings Using == and !=:

```
str1="apple"
str2="banana"

if [ "$str1" == "$str2" ]; then
    echo "Strings are equal"
else
    echo "Strings are not equal"
fi
```

The University of Lahore**String Comparison with -z (Check if the string is empty):**

```
str=""
if [ -z "$str" ]; then
    echo "String is empty"
else
    echo "String is not empty"
fi
```

String Comparison with -n (Check if the string is not empty):

```
str="Hello"
if [ -n "$str" ]; then
    echo "String is not empty"
else
    echo "String is empty"
fi
```

String Manipulation**Substring Extraction:**

```
str="Hello, world!"
echo ${str:7:5} # Outputs: world
```

Extracting a Substring from the End of a String:

```
str="Hello, world!"
echo ${str: -6} # Outputs: world!
```

String Replacement:

```
str="Hello, world!"
new_str=${str/world/universe} # Replaces 'world' with
'universe'
echo $new_str # Outputs: Hello, universe!
```

Remove Leading or Trailing Whitespaces:

```
str="  Hello, world!  "
trimmed_str=$(echo "$str" | xargs) # Removes leading and
trailing whitespaces
echo "$trimmed_str" # Outputs: Hello, world!
```

The University of Lahore**Advanced String Operations****String Splitting (Using IFS and read):**

```
str="apple,banana,orange"
IFS=',' read -ra arr <<< "$str" # Splits string by comma
into an array
for i in "${arr[@]}"; do
    echo $i # Outputs: apple, banana, orange
done
```

Using printf for String Formatting:

```
name="Alice"
age=25
printf "Name: %s\nAge: %d\n" "$name" "$age"
```

String Matching with Regular Expressions (using [[...]]):

```
str="Hello, world!"
if [[ "$str" =~ ^Hello ]]; then
    echo "String starts with 'Hello'"
else
    echo "String does not start with 'Hello'"
fi
```

String Case Conversion:

```
str="Hello, world!"
# Convert to uppercase
echo ${str^^} # Outputs: HELLO, WORLD!
# Convert to lowercase
echo ${str,,} # Outputs: hello, world!
```

Checking If a String Matches a Pattern:

```
str="abc123"

if [[ "$str" == abc* ]]; then
    echo "String starts with 'abc'"
fi
```

The University of Lahore**String Functions in Bash****Using tr to Transform Characters:**

```
echo "Hello, World!" | tr 'a-z' 'A-Z' # Converts to  
uppercase
```

Using sed for String Substitution:

```
str="Hello, world!"  
echo "$str" | sed 's/world/universe/' # Outputs: Hello,  
universe!
```

Using awk for String Manipulation:

```
str="apple banana cherry"  
echo "$str" | awk '{print $2}' # Outputs: banana
```

String Arrays**Declaring an Array of Strings:**

```
fruits=("apple" "banana" "cherry")  
echo ${fruits[0]} # Outputs: apple
```

Looping Through an Array of Strings:

```
fruits=("apple" "banana" "cherry")  
  
for fruit in "${fruits[@]}"; do  
    echo $fruit # Outputs: apple, banana, cherry  
done
```

Handling Multiline Strings**Multiline String Declaration:**

```
str="This is a  
multiline string  
in Bash."  
echo "$str"
```

Here Document (Heredoc) for Multiline Strings:

```
cat << EOF  
This is a  
multiline string  
using a here document.  
EOF
```

Practice Questions

- 1) Write a script to reverse a string using Bash string manipulation techniques.

```
GNU nano 7.2
#!/bin/bash

read -p "Enter a string: " str
reversed=""
len=${#str}

for (( i=len-1; i>=0; i-- )); do
    reversed="$reversed${str:$i:1}"
done

echo "Reversed string: $reversed"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que1.sh
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que1.sh
Enter a string: dracula
Reversed string: alucard
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 2) Write a script to find the index of a substring in a given string.

```
GNU nano 7.2
#!/bin/bash

str="Hello World"
substr="World"

index=$(expr index "$str" "$substr")
echo "Index of '$substr' in '$str' is $index"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que2.sh
Index of 'World' in 'Hello World' is 3
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

The University of Lahore

- 3) Write a script to check if one string contains another string.

```
GNU nano 7.2
#!/bin/bash

str="Hello World"
sub="World"

if [[ "$str" == *"$sub"* ]]; then
    echo "'$str' contains '$sub'"
else
    echo "'$str' does not contain '$sub'"
fi

abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$ nano que3.sh
abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$ chmod +x que3.sh
abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$ ./que3.sh
'Hello World' contains 'World'
abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$
```

- 4) Write a script to format and display a string that includes variables for a name and age.

```
GNU nano 7.2
#!/bin/bash

name="Abduallah"
age=19

echo "Hello, my name is $name and I am $age years old."

abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$ nano que4.sh
abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$ chmod +x que4.sh
abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$ ./que4.sh
Hello, my name is Abduallah and I am 19 years old.
abduallah-sohail@abduallah-sohail-VMware-Virtual-Platform:~$
```

- 5) Write a script that takes a comma-separated string and splits it into an array, then prints each item.

The University of Lahore

```
GNU nano 7.2
#!/bin/bash

csv="apple,banana,cherry"
IFS=',' read -ra arr << "$csv"

for item in "${arr[@]}"; do
    echo "$item"
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que5.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que5.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que5.sh
apple
banana
cherry
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

6) Write a script to count the number of lines in a file that contain a specific string.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="hello"

count=$(grep -c "$search" "$file")
echo "Number of lines containing '$search': $count"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que6.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que6.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que6.sh
Number of lines containing 'hello': 0
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

7) Write a script to extract all lines containing a specific string from a file and save them into a new file.

The University of Lahore

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="hello"
newfile="output.txt"

grep "$search" "$file" > "$newfile"
echo "Lines containing '$search' saved to $newfile"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que7.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que7.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que7.sh
Lines containing 'hello' saved to output.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

8) Write a script to replace a string in a file with another string.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="old"
replace="new"

sed -i "s/$search/$replace/g" "$file"
echo "Replaced '$search' with '$replace' in $file"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que8.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que8.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que8.sh
Replaced 'old' with 'new' in file.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

9) Write a script to print all the lines from a file that do not contain a specific string.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="hello"

grep -v "$search" "$file"
```


The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que9.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que9.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que9.sh
Hello!
I hate this world!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

10) Write a script to count and print the number of occurrences of a specific string in a file.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="hello"

count=$(grep -o "$search" "$file" | wc -l)
echo "The string '$search' occurs $count times in $file"

abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que10.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que10.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que10.sh
The string 'hello' occurs 0 times in file.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

11) Write a script to append a string to the end of a file.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
echo "This is a new line" >> "$file"
echo "Appended string to $file"

abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que11.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que11.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que11.sh
Appended string to file.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

12) Write a script to remove all empty lines from a file

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
sed -i '/^$/d' "$file"
echo "Removed all empty lines from $file"
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que12.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que12.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que12.sh
Removed all empty lines from file.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

13) Write a script to count the number of words in a file.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
words=$(wc -w < "$file")
echo "$file has $words words"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que13.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que13.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que13.sh
file.txt has 10 words
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

14) Write a script to search for a string in a file and print the line number of each occurrence.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="hello"

grep -n "$search" "$file"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que14.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que14.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que14.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

15) Write a script to delete a line from a file that contains a specific string.

```
GNU nano 7.2
#!/bin/bash

file="file.txt"
search="delete me"

sed -i "/$search/d" "$file"
echo "Deleted lines containing '$search' from $file"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano que15.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x que15.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./que15.sh
Deleted lines containing 'delete me' from file.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```